

---

# REPRESENTED STRUCTURE VERSUS REPRESENTATIONAL STRUCTURE: A CHALLENGE FOR INTERPRETING LLMs

---

Colin Klein

School of Philosophy  
The Australian National University  
colin.klein@anu.edu.au

## ABSTRACT

An adequate theory of representation should distinguish between the structure of a representation and the structure of what it represents. I argue that the simplest sorts of transformers (the architecture that underlies most familiar Large Language Models) have only a very lightweight structure for their representations: insofar as they work with the structure of language, they represent it but do not use it. In addition to being interesting in its own right, this also shows how we may use high-level invariants at the computational level to place constraints on representational formats at the algorithmic level.

## 1 Introduction

An adequate theory of representation should distinguish between the structure of a representation and the structure of what it represents. Subway maps represent the topological structure of the subway, but so does a written description. Same content, different *format*. Distinguishing represented structure from the structure of representation is an important special case of the general demand that one separate content from properties of the representation. Just as neural representations need not be coloured and subway maps need not smell of stale beer, the structure of a representation may have very different properties than the structure of its target. Heraldry gives a well-defined format for coats of arms, but there's no obvious resemblance between the addition of a *baton sinister* and illegitimate birth.

Representational format has played a key role in many debates in cognitive science. It also features in a number of recent philosophical debates about the distinction between perception and action (Block, 2023), whether cognition requires a language of thought (Quilty-Dunn et al., 2023), the modularity of spatial navigation (Duval, 2025), the role of language in thought (Camp, 2009), and many many others (Mollo and Vernazzani, 2023).

That the format of representations turns out to be important should not be a surprise: the computational theory of mind that animates cognitive science has always been sensitive to questions of format and data structure. As Marr and Nishihara (1978) pointed out, questions of algorithms and questions of format are complimentary. We can't really ask what an algorithmic process is doing without asking about the format of the representations it uses. Since many formats can be translated without loss into other formats (Haugeland, 1998), debates about format must occur at the level of what counts as simple or primitive operations (Sterelny, 1986). Format affects what a system can do only indirectly, via affecting how easy or hard it is to achieve certain tasks.

These debates have taken on renewed importance with the appearance of sophisticated machine learning systems, particularly transformer-based Large Language Models (LLMs). LLMs appear, to a reasonable

first approximation, to have the same sophisticated linguistic capabilities humans do. Yet it is not at all obvious that they *do* the same things as we do when we process language.

The present paper makes a modest contribution to that debate. Any discussions of representation in LLMs ought to be sensitive to the distinction between representational format and structured content. Facility of LLMs with language suggests, at a minimum, that they *represent* the structure of language. But it is far less clear whether they have representations that *use* the structures we use (or, indeed, any meaningful structure).

I'll be particularly concerned with a special case of this distinction, where a representation has as its content both the structure of the world and certain structured sequences of operations. An Ikea manual shows both the structure of a child's bed and also the sequence of operations it is necessary to perform to translate a pile of components into a bed. The manual does not itself build a bed (alas). Rather, you need a separate system (a beleaguered parent) that can consume these representations and translate the record of actions into the appropriate transformations in the world.

I will make the case that at least some LLMs (and more precisely, the simplest form of transformers) should be seen as representing structure and representing operations on that structure, rather than using structured representations. The argument is limited in scope, and is meant to be a proof of principle. In section 2 I give a simple example of how structure and operations can be represented in a relatively lightweight format. In section 3 I introduce the permutation invariance of simple transformers and the standard solution; in section 4 I then argue that we ought to think of simple transformers as merely representing structure. Section 5 addresses some objections by going more into detail about structured representations. I conclude in section 6 with some reflections about why structured representations seem to matter, and what the takeaway message might be for thinking about representation by brains.

## 2 The representational power of matrix multiplication

I begin with a toy example to show how one can represent both structure and operations with a relatively lightweight representational format. To begin, consider the classic empirical results on the mental rotation of three-dimensional objects (Shepard and Metzler, 1971), which show that the time taken to answer rotation questions is proportional to the angle of rotation. This suggests to many that the format of visual imagery is something like a picture or an icon; ensuing debates focus on whether the empirical data actually support such an interpretation (Pylyshyn, 2003). Note that in this debate everyone agrees (roughly) that the representational *content* involved: it is some representation of structured objects. The interesting debate is about *how* those objects are represented.

How might one deal with rotation tasks on a modern computer? Let's make the problem schematic and simpler: Suppose I have a set of points on a 2D plane, and the primary question I want to answer is whether one set of 2D points can be transformed by rotation to match another set of points. A natural way to start would be to represent my  $n$  points by storing them in an  $n \times 2$  matrix  $M$ , each row representing the  $x$  and  $y$  coordinate of one point.

Matrices are a nice choice for a format. They are the workhorse of most machine learning, because you can fit a lot of useful things in a matrix, and there are a lot of tools to work with matrices. Matrix multiplication also represents a generically and powerful medium for operating on representations. (To preempt a worry, the point is very much *not* going to be that machine learning uses matrix multiplication and matrix multiplication is too simple or unstructured or whatever to support interesting formats. The example is meant to pump intuitions only; subsequent sections will get into the nitty-gritty. )

So we have our representation of our points. We need an algorithm. Note that one way to achieve our task, if we had a bunch of transparent plastic, would be to make a picture of the array of dots and rotate it slowly, checking to see if we've matched our target, and returning a negative answer if we get all the way around.

We don't have transparent plastic, but it's easy enough to emulate this process with matrices. All I need to do is to pick a suitably small rotation angle  $\theta$  and then repeatedly multiply my matrix by the  $2 \times 2$  matrix

$$R = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

The result  $M = MR$  after each such operation is an array with the points rotated by  $\theta$  around the origin. I check, rinse, repeat.

Now, what is the *format* of  $M$ ? It is clearly a matrix, but is it anything more? Given how we’re using it, should we also call it a picture? (The two aren’t incompatible: I can use one format to implement another) I’m doing something that I can do with pictures, and my algorithm will have properties that the corresponding rotations on a picture would have (the number of steps taken to match two sets of points, for example, will be a function of the angular distance of rotation between them).

On the other hand, I can do all sorts of things to  $M$  that I could never do to a picture. I can warp and shear the location of the points. I can trivially make a higher-dimensional representation, projecting my  $x, y$  coordinates into a 14-dimensional space. I can do so using the very same process of matrix multiplication.

I think the more natural thing to say, at least in this case, is that just multiplying by a rotation matrix isn’t enough to make  $M$  picture-like. Instead, we can view  $M$  as representing a structure in the world (a set of points with a certain relationship) and  $R$  as a certain operation that can be applied to this structure (rotation). Matrix multiplication then applies the latter to the former.

The case is thus a mathematical analogue of the Ikea manual I mentioned in section 1: we have representations of structure and operations on those structures. The format of both is not pictorial. To actually execute the operations, though, we need a third operation (matrix multiplication) which is a generic way of applying the represented operation. Note that rotation is just one of the operations we might want to do on a set of points: we could have a whole series of transformations, represented by different matrices  $R', R''$ .... We could stitch these operations together in different useful sequences. The point is, again, that in doing so we must distinguish representations of operations on a represented structure from operations on a representation *with* structure.

### 3 Permutation invariance

Turn now to transformers (Vaswani et al., 2017), the architecture used by most of the familiar LLMs.<sup>1</sup> My argument concerns the very simplest form of a transformer. Technically speaking: I focus on simple encoder-only autoregressive transformers, and only those (like BERT Devlin, 2018) that do not mask the attention matrix (I return to more complex transformers in section 5).

I will be interested in particular of the format of what is called the *residual stream*, which is what carries the intermediate results of computations in the transformer. I focus on the residual stream because it is what carries all of the information that changes from run to run. This is in contrast to the weights, which are important for determining a transformer’s capacities but are fixed by training. In classic theories of (say) human syntactic processing, one distinguishes between core knowledge and the (say) the tree-like structures that are built up in the course of sentence production (?, Ch 3). The actions of the residual stream seem like the closest analogue to the latter, and the question at hand will be whether it has any sort of interesting structure.

My argument will be that transformers use a pretty unstructured format. If there is structure anywhere, then, it is in content, not the format of representations itself. To make the point I start with an unusual property of transformers known as *permutation invariance*. I introduce the property in this section and spell out the consequences in the next.

To input a string of text into a transformer, it must first be first tokenised into a string of  $t$  unique tokens and then embedded by linear projection into a  $t \times d$  matrix  $E$ . Each row of  $E$  thus corresponds to a token of the input, while  $d$  is the dimensionality of the embedding. Each block of a transformer takes the residual stream as input, uses this to modify each element, which gives a new  $t \times d$  matrix, which is then added back into the residual stream, normalised, and fed along to the next block (Millière and Buckner, 2024b, 11ff). At the end, it is decoded, and the last token is used as the completion of the original sentence.

---

<sup>1</sup>This is not meant to be a general introduction to transformers. There are plenty of good ones—see e.g. Vaswani et al., 2017; Alammari, 2018; Millière and Buckner, 2024b.

Here is a strange and under-appreciated fact about simple transformers. Transformer architectures are what is known as *permutation invariant*. Given a transformer  $T()$ , an input matrix  $X$ , and a permutation matrix  $P_\pi$ , then  $T(P_\pi X) = P_\pi T(X)$ . (See appendix A for a formal proof and more details.) In other words, if we feed a transformer some input and then permute the rows of the output, what we get is the same as if we permuted the rows of the input.

This is a remarkable property. It appears to imply the following: Suppose we start with “cats eat fish and” and  $T(X)$  gives the plausible next token “rats”. Let  $P_\pi$  swap the first and third rows of an input matrix. Since  $X$  encodes tokens in successive rows, and  $P_\pi X$  is a row-wise permutation, it follows that if we give  $T$  “fish eat cats and”, it should also return “rats”. Indeed, even a completely ungrammatical permutation will work: if we feed the transformer “and fish cats eat” it will return “rats”.

Of course, LLMs do not actually do this. It’s a good thing they don’t. *Language* is not permutation-invariant: the order of constituents matters!

The standard solution is to take the embedded input  $E$  and add to it a  $t \times d$  position matrix  $P$ .<sup>2</sup> The final input is then  $X = E + P$ , which is the initial value of the residual stream. Each row of  $P$  is distinct, which serves to add information about the position of each token in the original input.  $P$  itself does not vary from run to run, so the transformer has a reliable source of position information attached to each input token.<sup>3</sup>

The addition of the position encoding is often treated as a boring engineering detail not worth further commentary.<sup>4</sup> Note, however, that positional encoding still doesn’t change anything about permutation invariance. Rather, it adds information *about* position to the original embedding, making it available to the transformer to use in subsequent operations. Once position is added, you can still permute to your heart’s content: the basic operations of the transformer still won’t care.

## 4 The minimal structure of transformers

Permutation invariance is a property that appears at Marr (1982)’s computational level. It is an invariant of the function being computed by the transformer. Invariants at the computational level provide constraints on interpretations at the algorithmic level, which includes constraints on structure. Whatever structure we impute to the residual stream itself (and again, structure *of* the representation, not of what it represents) must be constrained by this fact.<sup>5</sup>

One straightforward consequence of permutation invariance, I suggest, is that we should think of the residual stream of transformers as having a relatively unstructured format. The residual stream is primarily a

<sup>2</sup>Positional encoding can in fact be done in many ways, each mathematically equivalent but differing in other important training-relevant properties (Dufter et al., 2022). I have focused on the simplest strategy, used by (Vaswani et al., 2017), but more complex strategies can be seen as differing implementations of this (Dufter et al., 2022). The choice of position matrix itself is also relatively unimportant for this argument, so long as it satisfies certain intuitive constraints (specifically monotonicity, translation invariance, and symmetry Wang et al., 2020)—informally, you want your position vectors to behave like distances. There also need to be relatively few token vectors  $w, v$  and position vectors  $a, b$  s.t.  $w + a \approx v + b$ . While one can find such collisions, this condition is (unsurprisingly) usually met.

<sup>3</sup>We can think of a positional encoding as adding additional information to the individual constituents of a set. So, as a crude example, one could represent our sample sentence above as:

$$\{\{\text{‘cats’}, 0\}, \{\text{‘eat’}, 1\}, \{\text{‘mice’}, 2\}, \{\text{‘and’}, 3\}\}$$

If you permute the elements of a representation of this set, information about ordering is still preserved. That is, the same sentence can be represented equally well as:

$$\{\{\text{‘mice’}, 2\}, \{\text{‘eat’}, 1\}, \{\text{‘cats’}, 0\}, \{\text{‘and’}, 3\}\}$$

(Because this is, of course, just the same set.)

<sup>4</sup>For example, in Millièrè and Buckner’s insightful two-part treatment of the state of the art in LLM interpretation, positional encoding is mentioned exactly once, in a figure caption (Millièrè and Buckner, 2024a, p11 fig 2).

<sup>5</sup>I take this to be an explanatory move similar to the emphasis on mathematical invariants of the natural world in the natural world. Many philosophers seem to think that physics is merely the science of the very tiny, something that has almost never been true (Klein, 2013, 2014). Invariance and symmetry principles in particular are key to many arguments in modern physics (van Fraassen, 1989). For a surprisingly accessible introduction to the role of such arguments in the making of our modern understanding of matter, see (Frisch, 1972).

collection of vectors. The basic operations performed on the residual stream are indifferent to the ordering of those vectors—indeed, one way to see why transformers are permutation-invariant is to note that each of the operations performed on the residual stream is indifferent to the ordering! There is no special significance to which vector is where in the residual stream, and no privileged relationship between any particular pair of vectors.<sup>6</sup>

That in turn means that the basic format used by the residual stream is extremely lightweight. Formally, it appears to be a kind of *multiset*.<sup>7</sup> That is, it is a container for a fixed number of objects, with no order, that behaves like a set except that there can be multiple copies of the same elements. Less formally, the residual stream is effectively a *sack*. It has the format you’d use to represent a bag of scrabble tiles (if scrabble tiles were 768-dimensional vectors). The operations performed on this sack fall into three categories: ones applied to each individual tile, ones that compare each tile to another sack of tiles, and ones that compare each of a sack of tiles to every other member. None of these really care about the order of the tiles. Permutation invariance is a consequence of the fact that each individual operation is indifferent to the ordering of vectors in the residual stream.

Of course, the entire discussion thus far has been about the format of the residual stream itself. Transformers obviously have competence with linearly structured information. The natural place to look to explain that capacity is not in the structure of the representation, but in the structure of what is represented—that is, in content rather than in format. Positional encoding adds information *about* structure to the content of a representation itself. The transformer also learns operations on the encoded information: that’s what’s embodied in the weights learned during training.

So I think we should say that transformers learn to *represent* structures without *using* them. They can also represent operations on structures. Recall the conclusion of section 2, where I argued that a rotation matrix is just the representation of a certain kind of transformation, with matrix multiplication acting to apply that transformation. That is a natural way to explain how LLMs gain a compact representation of a training corpus: rather than memorising everything (which they couldn’t do anyway), they must have some resources for taking what they do have and transforming it in various ways. The lesson is that these operations and applications can be done with a much weaker structure than (e.g.) a syntactic tree, or a set of sentences in the language of thought, or whatever.

To cut off a potential objection, I am not just saying that the residual stream is, when examined closely, too simple to support complex kinds of processing. That is, I am not committing what (Millière and Buckner, 2024a) call the ‘re-description fallacy’. As they put it

This fallacy arises when critics argue that a system cannot model a particular cognitive capacity, simply because its operations can be explained in less abstract and more deflationary terms. In the present context, the fallacy manifests in claims that LLMs could not possibly be good models of some cognitive capacity  $\phi$  because their operations merely consist in a collection of statistical calculations, or linear algebra operations, or next-token predictions. (Millière and Buckner, 2024a, 9–10)

I agree that this sort of move is too cheap. But the core of the argument here is the constraint put on *any* interpretation of format by the computational-level property of permutation independence. The details of transformers are illuminating insofar as they help explain why permutation independence obtains, and we could get to the same conclusion from the bottom-up, as it were. But it’s not a micro-level look at the algorithmic level that’s doing the work, but rather the macro-level constraints given by the computational level invariants.

<sup>6</sup>What is perhaps more surprising is that permutation invariance holds for the training phase as well as for the use of transformers. Xu et al. (2024) show that with a few additional constraints, permuted input can be trained just as well as ordinary input.

<sup>7</sup>I use typewriter font when I am discussing the formats of computational entities, rather than (e.g.) abstract mathematical objects like sets, or public formats like maps and grocery lists.

## 5 A bit more about structure

I have argued that simple transformers deal with the structure of their target domain without using particularly structured representations. The argument itself has been somewhat impressionistic. One might worry about this for a couple of reasons.

First, as I mentioned in section 1, the argument here applies only to transformers that don’t mask the attention matrix in order to make it lower triangular. Many interesting transformers (including the one in the canonical Vaswani et al., 2017 paper) do mask, and that’s an important part of their operation. Second, as a broader computational point, relatively structured formats can be built out of relatively unstructured formats. In the programming language Python, for example, a key-value `dict` is a relatively unstructured format, similar to a `set`. The Python package `networkx` (Hagberg et al., 2008) implements a `Graph`—which is quite structured—by building it (to a first approximation) out of a collection of `dicts`. Might not someone who wanted to defend the possibility that the residual stream has a complex format (rather than just representing structure) therefore argue that the residual stream is used in the same way, to implement a structured representation?

While a full story is beyond the scope of this paper,<sup>8</sup> I think saying a bit more about structure goes considerable way to showing how the first two can be addressed. Once we distinguish structured representations from their content, there’s a natural question about how to understand that structure in the first place. What does it mean, from an algorithmic point of view, to say that a structured bit of data is a `list`, rather than a `set`? These are algorithm-level questions, so the distinction is not to be found in the physical realisers of those items, or in general in details of implementation or storage. (Those details are also, and unsurprisingly, messy.)

Early computer scientists spent a lot of time thinking about what exactly it means to structure data. The ground of structure is what d’Imperio (1969, p1) calls it, “patterns of mutual accessibility” among elements of data.<sup>9</sup> As she puts it,

The data structures chosen by a human being in the solution of a problem are, essentially, sets of possible paths of accessibility among different elementary “atoms” of data. ... The determination that a certain data item should be directly accessible from certain others, and indirectly accessible in various degrees along paths among several others still, defines a *data structure* involving these data items. The choice of a data structure makes certain items immediate neighbors, others more distant “acquaintances”. (d’Imperio, 1969, p4)

Accessibility relationships determine how you traverse data structures. What is it to have a `list`-like structure? It is for each data element to stand in exploitable relationships to at most two others—the one in front and the one behind, if those exist—and in addition for those relationships to be unique (if *a* precedes *b*, it only precedes *b*, and so on). These accessibility relationships are one reason why there are important complementary relationships between data structures and the algorithms defined on them: for a great number of things you do to a `list`, you must *traverse* it in order. It is also why some things, like sorting, only make sense on some data structures: you can’t sort a `set` or a `Graph`: the former because there’s not enough structure, the latter because there is too much.

So return to the two concerns. To the second point, we can sharpen the initial claim a bit more. The reason why should say that simple transformers use a relatively unstructured format is precisely because the operations involved are either order-independent or all-to-all. There are no access restrictions placed between bits

<sup>8</sup>A full story would involve the combinations of the *operations* that a data structure makes available and the *guarantees* afforded by the basic operations definitive of an architecture (Klein, 2025). This distinction can be drawn at the algorithmic level, with the details specific to specific formats. Access relationships are a very important part of this story: they provide both restrictions on the operations and constraints on what can be guaranteed. To give a sense of the full complexity, however, one also has to allow for (e.g.) operations that don’t respect access relationships—most often in the initial creation of a structured bit of data, but also in cases like parallel operations on every aspect of `list`-like structures. While there is a lot of work to do there, access relationships are so fundamental to so many aspects of data structures that I think the basic point is secure.

<sup>9</sup>For remarks in a similar vein, see also Mealy (1967)’s discussion of ‘access functions’. Both references come via the historical discussion in (Knuth, 1997, §2.6), especially p462—Knuth, of course, now being the *locus classicus* for many basic distinctions between structured datatypes.

of data, and hence everything is “dealt with in the same way, like grain being ground in a mill.” (d’Imperio, 1969, p6) Creating more structure is a matter of restricting access: one makes a `list` out of a arbitrary blobs of memory by adding pointers and defining operations that respect those pointers. Transformers may *represent* further structure, but they don’t have obvious mechanisms to *respect* that structure in a way that would structure the representations themselves.

On the first point, we can make more precise why masked self-attention should be expected to have an effect, but also why that effect is surprisingly modest. Zeroing the upper triangular of the attention matrix is, by design, a way of restricting access relationships: it means that later tokens can only be influenced by earlier ones. This breaks permutation invariance, and so (by my lights) adds further structure to the residual stream. It also doesn’t add *much* structure, however. The result gets something that’s a bit like a `list`, though with more promiscuous access relationships. (A compact way to put the point is that masked transformers are still permutation invariant *with respect to earlier tokens*. What comes before a token matters, only the fact that it comes before the token). This is still much less structure than is typically proposed for cognitive operations like linguistic cognition, so masking doesn’t really add that much structure to the data. This suggests that any structural heavy lifting is still done by content, not format.

## 6 Why care about format, anyway?

I have argued that simple transformers do a lot with relatively little structure (and that even sophisticated transformers might not add much more). The lesson of section 2 was that there are ways of representing structure and operations on structure that do not require the representations themselves to have much structure. I think, as a loose analogy, this is what we ought to think about transformers too.

This is not meant to be a knock against transformers. Arguably, part of the success of transformers is their ability to representing lots of structure with comparatively unstructured representations. Structuring representations is a way to limit the amount of computation you have to do: to say something is (say) a Graph is to propose certain limits on what it makes sense to do to it. By contrast, the self-attention mechanism that is at the heart of transformers is, at its core, computationally promiscuous. It is a comparison of each member of the residual stream to each other member. How does a transformer know (say) what the referent of a particular pronoun in a sentence is? Answer: because it can check *everything*, because each word is checked against each other word. (That’s not all it does, of course. it’s a bit miraculous that transformers give you a way to do so much with so little. But the lack of structured data is an important part of the story.)

The state of the art before transformers can be seen as various attempts to impose structure of neural networks to capture the structure of the target domain. Convolutional deep neural nets for image processing eliminated all-to-all connectivity in order to allow the network to exploit spatial contiguity. Recurrent networks for language modelling present elements sequentially and use the effect of earlier computations to alter processing of later ones to get sensitivity to linear structure. These can all be seen as ways of structuring processing in order to capture structure of the target domain. The cost is that the resulting architecture is tuned to particular domains.

Transformers learn sensitivity to the structure of the target domain without baking that structure into the architecture—that is, without needing to use that structure to compute. So one way to explain the general success of transformers across domains is to say that they have given up on format in favour of representational content. This includes the ability to represent operations and structures that were baked into architecture in earlier models. So, for example, Cordonnier et al. (2019) argue that the early attention layers of image transformers do something very much like the initial convolution operations in convolutional deep neural nets. Rather than build in those convolutions as part of the *architecture*, though, they’re just one of many learned transformations.

With that in mind, return to the ideas about representational format that initially motivated the paper. If transformers only represent facts about structure (rather than using interesting structure), what does that mean for corresponding debates in cognitive science?

I can see three methodological conclusions one might draw—two cheap, one more interesting. I’ll mark the cheap ones quickly, and then elaborate on the more interesting one.

First, you could say that we cared about formats because we care about what humans do. We have some sense (through years of cognitive science) of the format of thought. It takes more than something that's multiset or even list-like. LLMs don't have that format. So they don't think: they're a parlour trick (Bender et al., 2021) or a neat way to compress information (Chiang, 2023) or whatever. Or, second, you could go for *modus tollens*. LLMs don't have a particularly complex format. Much of the argument for (say) the language of thought has been that thinking of cognition as having a certain format is the only (Fodor, 1975) or at least the best (Quilty-Dunn et al., 2023) game in town. The 'bitter lesson', in Sutton (2019)'s phrase, is that simple format and simple learning rules trained on sufficient data can do a better job than things we build ourselves.

I think there's a third possibility, though, and I conclude by trying to flesh it out a bit. The third route steps back and asks—in a non-rhetorical way—well, why did we ever care about formats? As I've noted above, the notion of a format really comes in from computer science, and reflects a fact about the limits of *engineering* as much as anything. When you get down to the level of the chip, at least in traditional architectures, you basically have unstructured registers and memory locations. Traditional chip architectures can only do relatively simple operations to each of these locations, so you need to do a lot of operations to do anything interesting.

Formats do two useful things in this context. First, insofar as there is data in the world that you want to keep track of, and the data itself requires doing similar things and keeping track of certain structured relationships, it's useful if the way you represent that data somehow reflects those relationships. The actual datatypes that have evolved—lists and strs and the like—represent design patterns that are generally useful for programmers as they keep track of the world.

Second, to do something useful with many simple operations, you need resources for managing control flow. In a lovely passage explaining microchips to readers in the 1980s, Kidder writes

How can the machine perform its tricks? The general answer lies in the fact that computer can follow conditional instructions. They can take two values and compare them—that comes down to simple arithmetic—and, if so commanded, can perform one action if the values are equal and another if they are not. In this ability to follow conditional instructions—an ability built into the machine—lies much of the computer's power. You can set before it, in sequence, bifurcating webs of conditional instructions, until the machine appears to make sophisticated decisions on its own. (Kidder, 1981, 89-90)

The complexity of managing data is not just keeping everything in the right place, but doing everything in the right order. Complex datatypes help manage the 'bifurcating webs of conditional instructions'; conversely, all that bifurcation is necessary because the basic datatypes are so thin that you need complex branching algorithms to do useful stuff. And, usefully, the sorts of formats that you come to when you try to capture data in the world *also* turn out to be very useful for keeping track of what you should do and when. A stack, say, is handy not just for representing certain kinds of queued data in the world, but also keeping track of where you need to return to when you're done working through nested subroutines.

Transformers are notable in the degree to which neither of the previous two motivations apply. On the first point: weights are learned, not designed. The cognitive limitations of humans and the trickiness of keeping data appropriately aligned doesn't matter. What structure is there (we might think) can be learned with enough data. The very basic datatype (vectors) is informationally rich. We can view the basic operations equivalently either as a simple two-place operation (the dot product) or as a series of very complex one-place instructions. On the latter view, a transformer has an extremely complex instruction set, one learned rather than engineered, and that allows for the effective manipulation of representational targets without going to the trouble of mirroring that structure in the format of the representation. On the second point: transformers are notable for being entirely feedforward architectures. There is no recurrence, no conditional branching. Each block of operations is applied, each time, regardless of the outcomes of the previous block.

That in turn suggests a more subtle way to get back to thinking about the format of human cognition and how it might or might not compare to transformers. Perhaps the focus should be on the very broad *computational* constraints on brains. That's obviously a big topic, and answers are all speculative. But I think brains split



the difference in interesting ways. Brains clearly do complex, high-dimensional processing of data, and artificial neural networks show how this might be done with far less structure than is involved in traditional computational theories (Buckner, 2018).

On the other hand, brains are under very strong energetic constraints (Sterling and Laughlin, 2015), which might make the economy of processing afforded by formats more attractive. And control flow is clearly important in brains, especially in subcortical structures. In the evolutionary history of brains, there are major transitions (Klein, 2022; Barron et al., 2023)—like the development of recurrence, or lamination of processing—which can be understood as the development of more complex strategies for control flow. Humans also have the ability to (e.g.) follow rules, which involves the use of represented content to effect low-level flow-control.

So there is, alas, no clean answer as to the comparison with humans. But the broader point is (I hope) still illuminating. There is a relationship between the format of representation and the broad computational architecture in which that representation is embedded. The question is less whether transformers work like humans, and more about how we can use the relationship between format and architecture to illuminate the constraints on any complex computational system.

## Acknowledgements

Thanks to audiences at NYU, University of Pittsburgh HPS, Washington University St. Louis, Johns Hopkins University, CUNY Grad Center, and ANU for feedback on an ancestor of this paper. Special thanks to Nuhu Osman Attah, Andrew Barron, Ned Block, Cameron Buckner, Carl Craver, David Chalmers, Alexandre Duval, EJ Green, Sarah-Jane Leslie, Mei Liou, Edouard Machery, Eric Mandelbaum, and Ian Phillips for helpful discussion and feedback. This work was supported by TWCF-2020-20539 and ARC DP240100400.

## A Proof of Permutation Invariance

The following is based closely on the proof given by Dufter et al. (2022). Each transformer pass consists of a series of a self-attention block followed by a fully connected multilayer perceptron block, along with various norming and activation functions. As per the main text, I am considering only simple transformers that do not mask the attention matrix.

Let  $P_\pi$  be a permutation matrix and  $X$  an input. Any element-wise function (including activation functions like *ReLU*) is trivially permutation-invariant. Any row-wise function is also trivially permutation invariant; this includes norming layers.

Each layer of a fully connected multilayer perceptron layer is permutation invariant. A layer is equivalent to  $XW$ , where  $W$  is a weight matrix. Since matrix multiplication is associative,  $(P_\pi X)W = P_\pi(XW)$ .

It thus remains to be proven that the attention block is permutation invariant. Note that while one often speaks of multiple attention heads, in the forward pass these are equivalent to (and often implemented by) a single set of matrix multiplications, and I treat them as such.

Given an input  $X$ , the output of the attention head  $M$  can be decomposed into two steps:

$$\begin{aligned} A(X) &= XW^{(q)}(XW^{(k)})^\top \\ &= XW^{(q)}W^{(k)\top}X^\top \\ M(X) &= \text{softmax}(A)XW^{(v)} \end{aligned}$$

Where  $W^{(q)}$ ,  $W^{(k)}$ , and  $W^{(v)}$  are, respectively, the query, key, and value weights. Since we have accounted for the other operations, permutation invariance overall holds iff  $M(P_\pi X) = P_\pi M(X)$ .

Consider first the effect on  $A$  of permuted input:

$$\begin{aligned} A(P_\pi X) &= P_\pi XW^{(q)}(P_\pi XW^{(k)})^\top \\ &= P_\pi XW^{(q)}W^{(k)\top}(P_\pi X)^\top \\ &= P_\pi XW^{(q)}W^{(k)\top}X^\top P_\pi^\top \\ &= P_\pi AP_\pi^\top \end{aligned}$$

Now,  $\text{softmax}(P_\pi AP_\pi^\top) = P_\pi \text{softmax}(A)P_\pi^\top$ : the left multiplication gives a row permutation, and since softmax is row-wise it doesn't matter whether you permute rows before or after. The right multiplication gives a column permutation, and since softmax only cares about the weight of an element as a function of the transformed sum of the rows, not the position of the element in the row, a column-wise permutation can happen before or after softmax as well.

For any permutation matrix,  $P_\pi^\top P_\pi = I$ . So it follows that

$$\begin{aligned} M(P_\pi X) &= P_\pi \text{softmax}(A)P_\pi^\top P_\pi XW^{(v)} \\ &= P_\pi \text{softmax}(A)XW^{(v)} \end{aligned}$$

hence  $M(P_\pi X) = P_\pi M(X)$  ■

## References

- Alammar, J. (2018). The illustrated transformer. <https://jalammar.github.io/illustrated-transformer/>.
- Barron, A. B., Halina, M., and Klein, C. (2023). Transitions in cognitive evolution. *Proceedings of the Royal Society B: Biological Sciences*, 290(2002):20230671.
- Bender, E. M., Gebru, T., McMillan-Major, A., and Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*, pages 610–623.
- Block, N. (2023). *The border between seeing and thinking*. Oxford University Press.
- Buckner, C. (2018). Empiricism without magic: transformational abstraction in deep convolutional neural networks. *Synthese*, 195(12):5339–5372.
- Camp, E. (2009). A language of baboon thought. In Lurz, R. W., editor, *The philosophy of animal minds*, pages 108–127. Cambridge University Press, Cambridge.
- Chiang, T. (2023). ChatGPT is a blurry JPEG of the web. *The New Yorker* Feb 9.
- Cordonnier, J.-B., Loukas, A., and Jaggi, M. (2019). On the relationship between self-attention and convolutional layers. *arXiv preprint arXiv:1911.03584*.
- Devlin, J. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.
- d’Imperio, M. E. (1969). Data structures and their representation in storage. *Annual Review in Automatic Programming*, 5:1–75.
- Dufter, P., Schmitt, M., and Schütze, H. (2022). Position information in transformers: An overview. *Computational Linguistics*, 48(3):733–763.
- Duval, A. (2025). In defense of language-independent flexibility, or: What rodents and humans can do without language. *Mind & Language*, 40(1):93–119.
- Fodor, J. (1975). *The Language of Thought*. Thomas Y. Crowell Company, New York.
- Frisch, O. R. (1972). *The Nature of Matter*. Thames and Hudson, New York.
- Hagberg, A., Swart, P. J., and Schult, D. A. (2008). Exploring network structure, dynamics, and function using NetworkX. Technical report, Los Alamos National Laboratory (LANL), Los Alamos, NM (United States).
- Haugeland, J. (1998). Representational genera. In *Having Thought: Essays in the Metaphysics of Mind*, pages 171–206. Harvard University Press, Cambridge MA.
- Kidder, T. (1981). *The soul of a new machine*. Back Bay, New York.
- Klein, C. (2013). Multiple realizability and the semantic view of theories. *Philosophical Studies*, 163(3):683–695.
- Klein, C. (2014). Psychological explanation, ontological commitment, and the semantic view of theories. In Sprevak, M. and Kallestrup, J., editors, *New Waves in Philosophy of Mind*, pages 208–225. Palgrave Macmillan, New York.
- Klein, C. (2022). Explaining neural transitions through resource constraints. *Philosophy of Science*, 89(5):1196–1202.

- Klein, C. (2025). Computational individuation: Isomorphism, not indeterminacy. *Analysis (forthcoming)*, <https://doi.org/10.1093/analys/anaf003>.
- Knuth, D. E. (1997). *The Art of Computer Programming Volume 1: Fundamental Algorithms*, volume 1. Addison–Wesley, Reading, Massachusetts, 3rd edition.
- Marr, D. (1982). *Vision: A computational investigation into the human representation and processing of visual information*. WH Freeman, New York.
- Marr, D. and Nishihara, H. K. (1978). Representation and recognition of the spatial organization of three-dimensional shapes. *Proceedings of the Royal Society of London. Series B. Biological Sciences*, 200(1140):269–294.
- Mealy, G. H. (1967). Another look at data. In *Proceedings of the November 14-16, 1967, fall joint computer conference*, pages 525–534.
- Millière, R. and Buckner, C. (2024a). A philosophical introduction to language models–part I: Continuity with classic debates. *arXiv preprint arXiv:2401.03910*.
- Millière, R. and Buckner, C. (2024b). A philosophical introduction to language models–part II: The way forward. *arXiv preprint arXiv:2405.03207*.
- Mollo, D. C. and Vernazzani, A. (2023). The formats of cognitive representation: A computational account. *Philosophy of Science*, pages 1–20.
- Pylyshyn, Z. (2003). Return of the mental image: Are there really pictures in the brain? *Trends in cognitive sciences*, 7(3):113–118.
- Quilty-Dunn, J., Porot, N., and Mandelbaum, E. (2023). The best game in town: The reemergence of the language-of-thought hypothesis across the cognitive sciences. *Behavioral and Brain Sciences*, 46:e261.
- Shepard, R. N. and Metzler, J. (1971). Mental rotation of three-dimensional objects. *Science*, 171(3972):701–703.
- Sterelny, K. (1986). The imagery debate. *Philosophy of Science*, 53(4):560–583.
- Sterling, P. and Laughlin, S. (2015). *Principles of neural design*. MIT Press, Cambridge.
- Sutton, R. (2019). The bitter lesson. *Incomplete Ideas (blog)*, 13(1).
- van Fraassen, B. (1989). *Laws and Symmetry*. Oxford University Press, New York.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30:6000–6010.
- Wang, B., Shang, L., Lioma, C., Jiang, X., Yang, H., Liu, Q., and Simonsen, J. G. (2020). On position embeddings in bert. In *International Conference on Learning Representations*.
- Xu, H., Xiang, L., Ye, H., Yao, D., Chu, P., and Li, B. (2024). Permutation equivariance of transformers and its applications. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5987–5996.